System Architecture: UML Sequence Diagrams

All three diagrams follow the UML sequence diagram notation from the course reference (Microsoft Visio, "Create a UML sequence diagram"):

- **Actor lifeline** (stick figure): the external client. **Object lifelines**: the master thread, the request buffer and the worker threads.
- **Solid arrow**: a call or message. **Dashed arrow**: a return. **Open arrowhead**: an asynchronous message (condition-variable signals, which do not wait for a reply).
- **Self message**: work a component does on its own. **Activation bar**: when a component is actively running.
- **loop** and **alt** fragments: repeated steps and if/else branches.

The diagrams describe the server we are building (OSTEP Project G.5): a master thread, a bounded request buffer protected by one mutex and two condition variables (`not_empty`, `not_full`), and a fixed-size pool of worker threads. They focus on the concurrency and synchronization paths, so the details of how a worker reads files or runs CGI programs are left out.

Diagram 1: Handling one HTTP request (FIFO)

This is the main critical path of the server, executed once for every request. The master thread only accepts the connection, takes the buffer mutex, puts the connection into the bounded buffer, signals `not_empty` and goes straight back to `accept()`. A worker takes the oldest request out, signals `not_full`, then reads, parses and answers the request. The hot spot is the buffer mutex, which every request takes twice (once to enqueue, once to dequeue). The timestamps recorded here (`t_arrive`, `t_dequeue`, `t_done`) are what we use to measure queueing delay and latency.

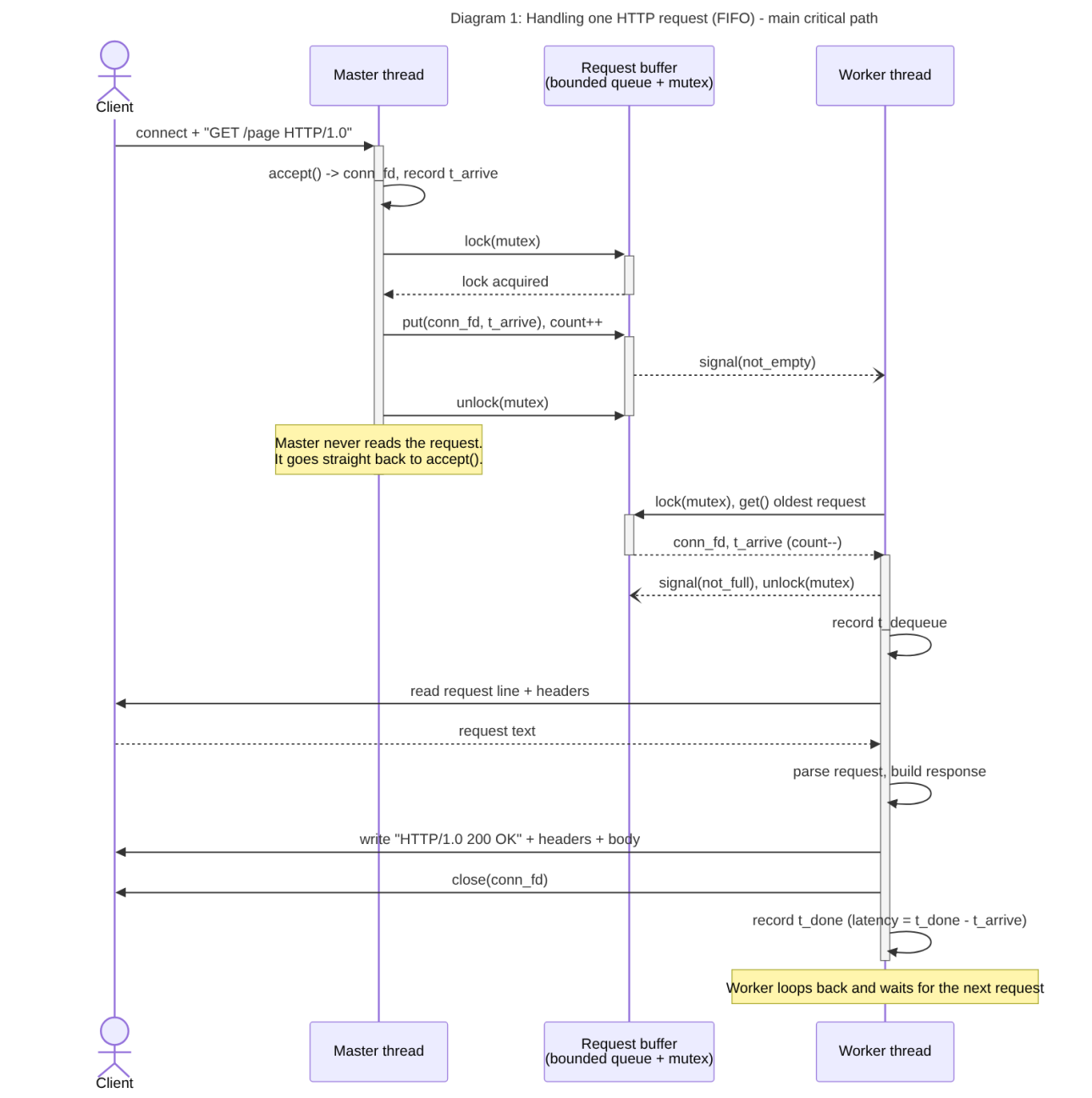


Diagram 2: Server startup, idle workers waiting, and wake-up

At startup the master creates the bounded buffer and a fixed number of worker threads (`-t`). With no work, each worker locks the mutex and calls `wait(not_empty)` inside a `while (count == 0)` loop, which releases the mutex and puts the thread to sleep, so there is no busy-waiting. When a request arrives, the master's `signal(not_empty)` wakes exactly one worker. The `while` loop matters because a woken thread has to re-check the condition before it takes a request.

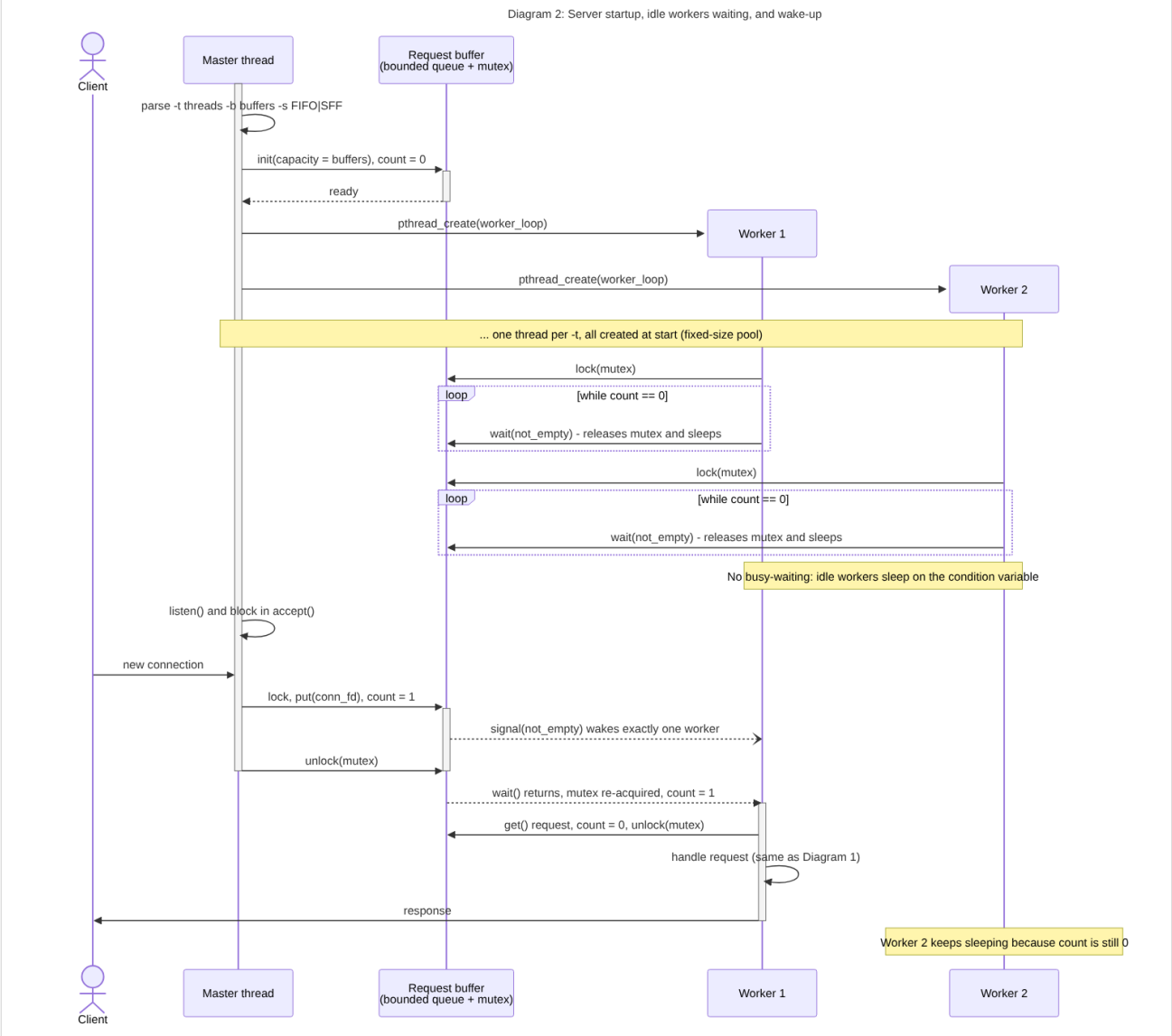
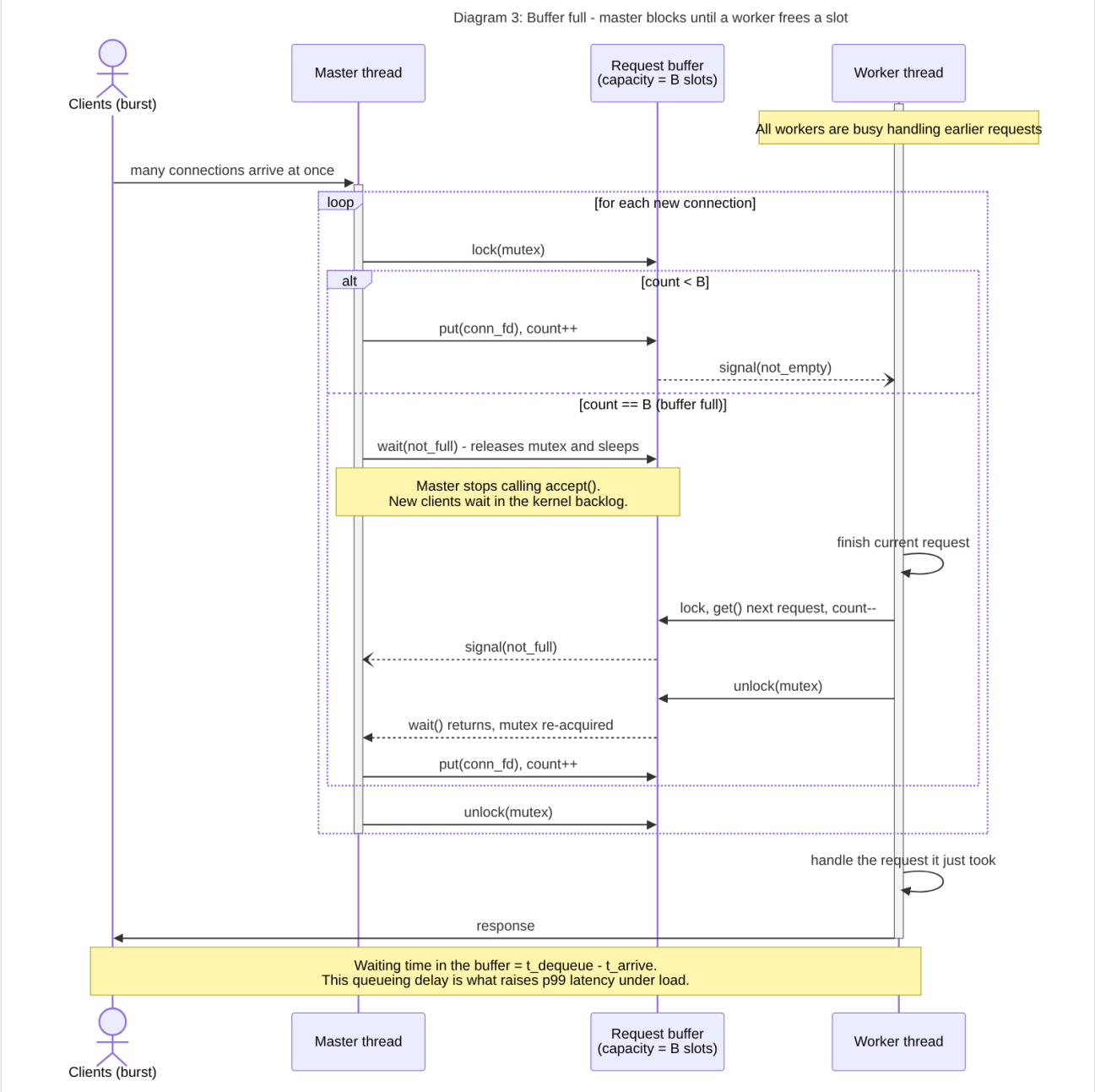


Diagram 3: Buffer full, the master blocks (backpressure)

When requests arrive faster than the workers can finish them, the buffer fills up. The `alt` fragment shows the two cases. If there is a free slot, the master enqueues as usual. If `count == B`, the master calls `wait(not_full)` and stops accepting connections, so new clients pile up in the kernel's accept backlog. As soon as a worker takes a request and signals `not_full`, the master wakes up and continues. The time a request spends here (`t_dequeue - t_arrive`) is the queueing delay that drives p99 latency up under load, which is exactly what our experiments will measure.

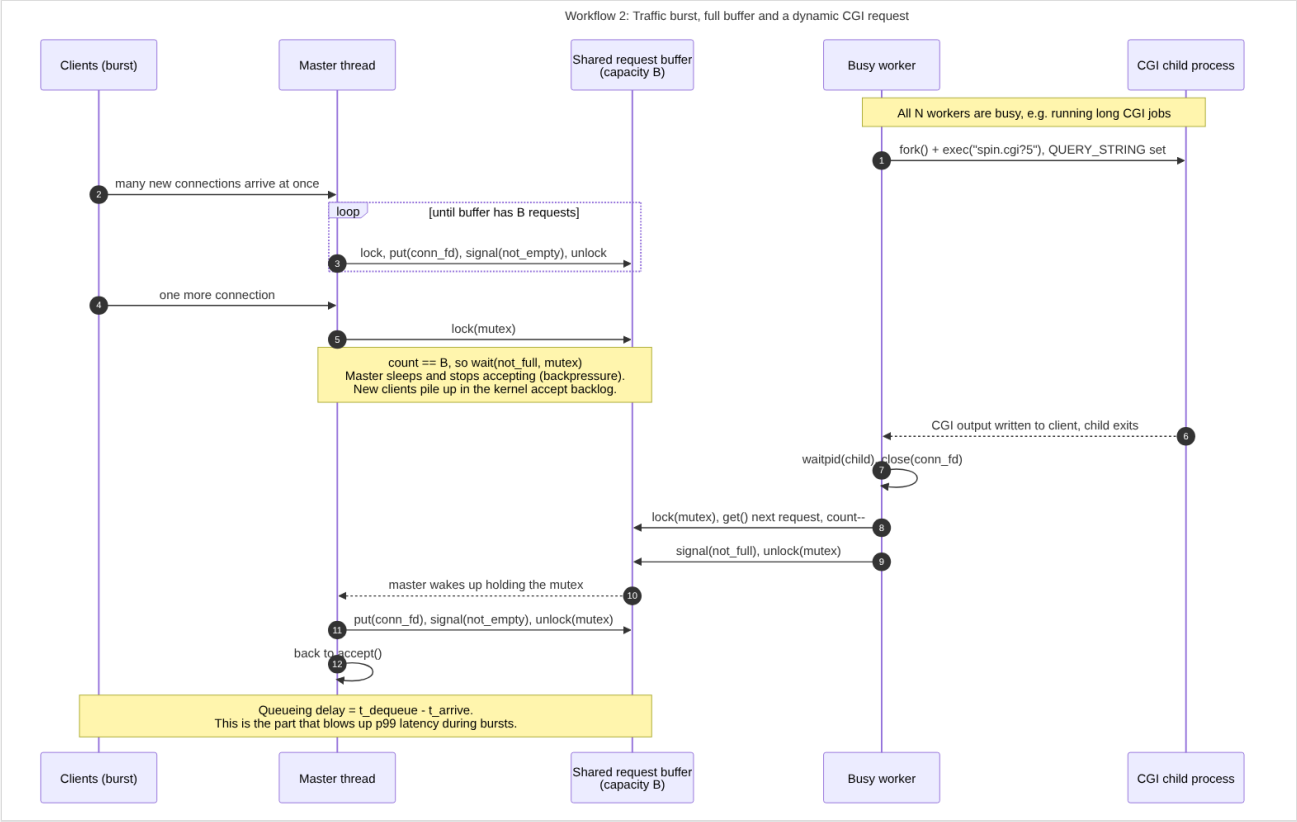


Appendix: Extension Cases

These two diagrams are not part of the core design above. They sketch possible extensions we may build later, depending on how the baseline turns out.

Extension A: Burst with long-running CGI requests

The same backpressure as Diagram 3, but the workers are busy with long dynamic (CGI) requests that use `fork()` / `exec()` / `waitpid()`, which makes the buffer fill up much faster.



Extension B: Adaptive thread pool (research extension)

A monitor thread samples the queue length and grows or shrinks the worker pool between a minimum and a maximum size, retiring idle workers safely between requests. This is the mechanism described in our research proposal.

